THIẾT KẾ VÀ ĐÁNH GIÁ HỆ THỐNG GIA SƯ THÔNG MINH DỰA TRÊN MODEL CONTEXT PROTOCOL MCP HỖ TRỢ HỌC LẬP TRÌNH PYTHON

Trần Khánh

Tóm tắt. Báo cáo trình bày thiết kế PyTutor, một hệ thống gia sư thông minh trực tuyến hỗ trợ sinh viên luyện lập trình Python bằng bài tập, kiểm thử và gợi ý theo từng bước. Model Context Protocol (MCP) được đề xuất làm lớp giao tiếp giữa ứng dụng gia sư và các công cụ như chạy mã, kiểm thử và phân tích cú pháp; các quyết định sư phạm thuộc về mô hình người học, quy tắc trợ giúp và mô hình ngôn ngữ. Báo cáo phân biệt hai nguyên mẫu hiện có: một demo trao đổi MCP thực qua stdio với công cụ run_code, và một demo luồng gia sư dùng các hàm công cụ mô phỏng để tạo kết quả kiểm thử, phân tích AST và gọi dịch vụ Qwen. Vì hai phần chưa được tích hợp thành một hệ thống hoàn chỉnh, kết quả hiện tại chỉ chứng minh tính khả thi ở mức giao thức và luồng minh họa; chưa chứng minh hiệu quả học tập hoặc độ an toàn khi chạy mã không tin cậy.

Từ khóa: hệ thống gia sư thông minh; MCP; học lập trình; Python; gợi ý thích ứng

1 Giới thiệu

Người mới học lập trình thường nhận được thông báo lỗi hoặc kết quả sai mà chưa biết cách lần theo nguyên nhân. Một chatbot chỉ nhìn đoạn mã có thể giải thích cú pháp, nhưng không nhất thiết biết chương trình đã chạy ra sao trên các trường hợp kiểm thử. PyTutor hướng đến một không gian làm bài trong đó sinh viên có thể đọc đề, viết mã, chạy thử, xem kết quả và yêu cầu gợi ý mà không phải chuyển ngữ cảnh sang một công cụ khác.

Mục tiêu của đề tài là thiết kế một gia sư lập trình Python có thể kết hợp dữ liệu bài tập, mã nguồn, kết quả thực thi và lịch sử làm bài để hỗ trợ quá trình sửa lỗi. MCP được khảo sát và thử nghiệm như giao thức tích hợp giữa ứng dụng chủ và các công cụ. Phạm vi hiện tại gồm wireframe chức năng chính, nguyên mẫu giao thức cục bộ và nguyên mẫu minh họa luồng tạo hint. Báo cáo không xem các nguyên mẫu này là một sản phẩm đã triển khai đầy đủ hoặc một nghiên cứu đã đo lường kết quả học tập.

2 Cơ sở và công nghệ liên quan

2.1 Hệ thống gia sư thông minh

Một ITS cần phản hồi theo tiến trình giải bài và có thể cập nhật hiểu biết về trạng thái người học. Mô hình tutoring của VanLehn mô tả vòng tương tác trong lúc giải từng

bước, nơi hệ thống có thể đưa phản hồi, gợi ý và cập nhật Student Model [4]. Các ITS dạy lập trình khác nhau về phạm vi kiến thức, hình thức phản hồi và tính năng hỗ trợ; vì vậy việc có một LLM hoặc compiler riêng lẻ chưa đủ để khẳng định một ứng dụng là gia sư thích ứng [5].

2.2 Vai trò của MCP

MCP định nghĩa mô hình Host-Client-Server dựa trên JSON-RPC. Host là ứng dụng điều phối; Host tạo MCP Client để kết nối tới Server; Server cung cấp các khả năng đã khai báo. Các primitive chính gồm Tools để gọi hành động, Resources để đọc dữ liệu và Prompts để cung cấp mẫu hướng dẫn [1], [2]. Client có thể liệt kê Tools bằng tools/list rồi gọi một Tool bằng tools/call. Giao thức không tự chạy chương trình, tạo dữ liệu đáng tin cậy hay quyết định cách dạy; các việc đó thuộc về phần mềm thực thi và logic của ứng dụng.

MCP hỗ trợ vận chuyển qua stdio cho tiến trình cục bộ và Streamable HTTP cho kết nối từ xa [3]. Nguyên mẫu trong dự án dùng stdio. Nếu chuyển sang dịch vụ web, cần triển khai và bảo vệ endpoint từ xa, bao gồm xác thực và kiểm soát nguồn truy cập. MCP có thể chuẩn hóa cách Host kết nối nhiều công cụ, nhưng bản thân giao thức không đảm bảo hệ thống sẽ chính xác hơn hoặc dễ mở rộng hơn trong mọi thiết kế.

3 Thiết kế hệ thống PyTutor

3.1 Người dùng và chức năng chính

Đối tượng sử dụng chính là sinh viên đang học Python. Trong một bài tập, sinh viên đọc yêu cầu và ví dụ, chỉnh sửa mã trong trình soạn thảo, chọn Chạy thử để xem output hoặc chọn Nộp bài để đối chiếu với bộ kiểm thử. Khi chưa tìm được nguyên nhân, sinh viên yêu cầu gợi ý; hệ thống nên chỉ đưa thông tin vừa đủ để người học tiếp tục tự sửa mã.

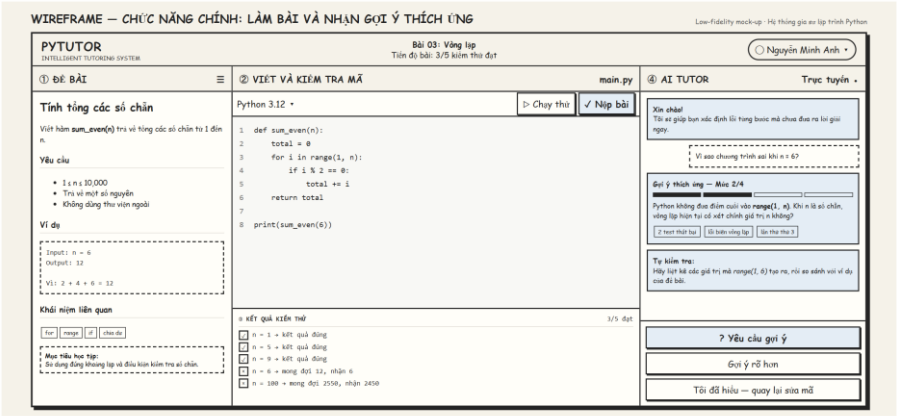
3.2 Kiến trúc đề xuất

Giao diện trình duyệt gửi thao tác tới backend PyTutor, đóng vai trò MCP Host. Host quản lý phiên làm bài, lấy ngữ cảnh cần thiết, áp dụng quy tắc sư phạm và gọi dịch vụ mô hình ngôn ngữ. MCP Client trong Host kết nối tới MCP Server. Server có thể cung cấp các Tool như run_tests, analyze_ast và các Resource như nội dung đề bài hoặc lịch sử nộp bài. Phần mềm kiểm thử, cơ sở dữ liệu và môi trường chạy mã là các thành phần riêng phía sau Server; MCP chỉ quy định cách chúng được cung cấp qua giao thức.

Khi sinh viên yêu cầu hint, Host ghép đề bài, mã hiện tại, kết quả kiểm thử, lỗi phân tích và các hint trước đó nếu có. Student Model lưu thông tin tiến trình; Pedagogical Model chọn mức trợ giúp; mô hình ngôn ngữ diễn đạt phản hồi. Các thành phần Student Model, Pedagogical Model, cơ sở dữ liệu và tích hợp MCP đầy đủ thuộc kiến trúc mục tiêu, chưa được chứng minh là đã hoàn thiện trong các demo hiện có.

3.3 Wireframe chức năng chính

Wireframe minh họa một màn hình gồm đề bài, trình soạn thảo và kết quả kiểm thử, cùng khu vực AI Tutor. Người học có thể yêu cầu gợi ý hoặc yêu cầu giải thích rõ hơn. Đây là bản phác thảo giao diện, không phải ảnh chụp một hệ thống đã triển khai.



Hình 1. Wireframe chức năng làm bài và nhận gợi ý thích ứng

3.4 Luồng yêu cầu gợi ý

Bước 1 - Thu thập ngữ cảnh. Host nhận mã hiện tại, đề bài, kết quả chạy hoặc kiểm thử và trạng thái bài làm.

Bước 2 - Gọi công cụ. MCP Client gọi các Tool phù hợp; Server trả kết quả có cấu trúc để Host kiểm tra.

Bước 3 - Chọn cách trợ giúp. Pedagogical Model chọn mức gợi ý, sau đó LLM tạo phản hồi ngắn dựa trên bằng chứng.

Bước 4 - Người học tiếp tục. Sinh viên xem hint, sửa mã và chạy lại; kết quả được ghi nhận để dùng trong vòng hỗ trợ kế tiếp.

Wireframe hiện thể hiện thanh gợi ý bốn mức, trong khi demo luồng gia sư dùng ba cấu hình prompt. Trước khi tích hợp giao diện và backend, cần thống nhất số mức và nội dung của từng mức. Một cách chia phù hợp là: gợi mở kiến thức; chỉ vùng cần xem xét; giải thích nguyên nhân; đưa ra các bước sửa hoặc mã giả, không cung cấp ngay lời giải hoàn chỉnh. Đây là đề xuất thiết kế, chưa phải kết quả đánh giá với người học.

4 Nguyên mẫu và mức độ triển khai hiện tại

4.1 Demo giao thức MCP thực

Trong mcp_visual_demo.py, Streamlit đóng vai trò giao diện demo và MCP Client dùng Python SDK kết nối qua stdio tới mcp_server_pytutor.py. Server khai báo một

Tool là run_code. Client khởi tạo phiên, liệt kê Tool và gọi run_code; phần log trong giao diện được dựng lại từ kết quả SDK để minh họa các bước, không phải bản ghi thô đọc trực tiếp từ kênh stdio. Đây là phần hiện thực hóa MCP thực trong thư mục dự án. Tool run_code thực thi Python bằng exec trong tiến trình Server; việc chạy ở tiến trình con không đồng nghĩa với sandbox bảo mật. Demo này chưa kết nối LLM, cơ sở dữ liệu hay Student Model.

4.2 Demo luồng gia sư

Trong pytutor_mcp_demo.py, giao diện cho thấy bài tập tính tổng, các ca kiểm thử, phân tích AST và các mức yêu cầu gợi ý. Hàm Python cục bộ thực hiện run_tests và analyze_ast; chương trình tự tạo các đối tượng JSON để trình bày log giống thông điệp MCP, nhưng nhánh này không dùng MCP Client/Server của SDK để trao đổi các Tool đó. Ứng dụng có đoạn gọi một API Qwen bên ngoài. Do đó, đây là demo luồng tương tác và prompt, không phải bằng chứng rằng các Tool tutoring đã được tích hợp qua MCP.

5 Kết quả và thảo luận

Hai nguyên mẫu chứng minh được hai việc riêng: demo giao thức cho thấy Client có thể kết nối tới Server cục bộ, khám phá Tool và nhận kết quả gọi Tool; demo gia sư cho thấy có thể đưa kết quả kiểm thử và nhận xét AST vào prompt để tạo phản hồi theo mức. Chưa có một luồng duy nhất dùng MCP thực từ giao diện sinh viên tới các Tool kiểm thử, phân tích và dịch vụ LLM. Vì vậy, kết luận phù hợp hiện nay là tính khả thi kỹ thuật ở mức nguyên mẫu, không phải mức hoàn thiện của sản phẩm.

Bản v2 ghi nhận thời gian xấp xỉ 1 ms khi chạy trực tiếp và khoảng 300-500 ms qua demo MCP. Các con số này phụ thuộc máy thử nghiệm, thời gian khởi động tiến trình con và cách đo; tài liệu chưa nêu đủ số lần lặp và điều kiện đo để xem đây là benchmark. Chúng cũng không so sánh công bằng hai hệ thống tương đương: MCP bổ sung khâu giao thức và khởi động, còn chạy trực tiếp bỏ qua các khâu đó. Không thể suy ra từ số liệu này rằng MCP luôn chậm như vậy hoặc một server chạy liên tục sẽ có độ trễ cụ thể nào.

6 Hạn chế và kế hoạch đánh giá

Hạn chế lớn nhất là hai demo chưa được nối thành một kiến trúc MCP thống nhất. Các mã chạy bằng exec không bị cô lập khỏi hệ điều hành bằng container hay cơ chế sandbox; vì thế không nên dùng chúng để chạy mã sinh viên trên máy chủ dùng chung. Trước khi thử nghiệm với người dùng, cần cô lập tiến trình và áp dụng giới hạn thời gian, CPU, bộ nhớ, tệp và mạng. Cần bổ sung kiểm soát đầu vào, xác thực, quyền truy cập và ghi nhật ký phù hợp.

Giai đoạn kỹ thuật tiếp theo là đưa run_tests và analyze_ast vào MCP Server, kết nối bằng MCP Client thật, xác nhận schema đầu vào/đầu ra và kiểm tra các ca thành công,

lỗi cú pháp, timeout cùng lời gọi Tool không hợp lệ. Sau đó mới tích hợp Student Model, lưu lịch sử và quy tắc chọn hint. Khi triển khai qua mạng, có thể chuyển sang Streamable HTTP với xác thực và các kiểm soát bảo mật của transport [3].

Đánh giá nên tách thành ba lớp: (1) kiểm tra đúng giao thức và độ tin cậy của lời gọi Tool; (2) đánh giá chất lượng hint bằng rubric, gồm tính chính xác, mức độ gợi mở và việc tránh lộ lời giải; (3) nghiên cứu người học để xem hệ thống có hỗ trợ hiểu bài và tự sửa lỗi hay không. Chỉ lớp thứ ba mới cung cấp bằng chứng về hiệu quả học tập; cần thiết kế hoạt động, tiêu chí và quy trình thu thập dữ liệu trước khi đưa ra kết luận.

7 Kết luận

PyTutor đề xuất kết hợp giao diện làm bài Python, kiểm tra tự động và gợi ý thích ứng; MCP đảm nhiệm lớp giao tiếp để Host sử dụng dữ liệu và công cụ. Mã nguồn dự án hiện có một demo MCP thực cho Tool run_code và một demo gia sư riêng sử dụng các hàm kiểm thử, phân tích AST cục bộ cùng lời gọi Qwen. Việc phân biệt rõ hai nguyên mẫu giúp xác định bước hoàn thiện tiếp theo: tích hợp Tool tutoring qua MCP, xây môi trường thực thi an toàn, thống nhất mức gợi ý và đánh giá hệ thống bằng kiểm thử kỹ thuật lẫn nghiên cứu với người học.

8 Tài liệu tham khảo

[1] Model Context Protocol, “Architecture,” MCP Specification, 2025-06-18. https://modelcontextprotocol.io/specification/2025-06-18/architecture

[2] Model Context Protocol, “Tools,” MCP Specification, 2025-06-18. https://modelcontextprotocol.io/specification/2025-06-18/server/tools

[3] Model Context Protocol, “Transports,” MCP Specification, 2025-06-18. https://modelcontextprotocol.io/specification/2025-06-18/basic/transports

[4] K. VanLehn, “The Behavior of Tutoring Systems,” International Journal of Artificial Intelligence in Education, vol. 16, no. 3, pp. 227-265, 2006. doi:10.3233/IRG-2006-16(3)02.

[5] T. Crow, A. Luxton-Reilly, and B. Wünsche, “Intelligent Tutoring Systems for Programming Education: A Systematic Review,” in Proceedings of the 20th Australasian Computing Education Conference, pp. 53-62, 2018. doi:10.1145/3160489.3160492.